

Grafica e interfacce

Interfacce grafiche con Tkinter

Creare finestre e interfacce grafiche semplici con Tkinter in Python.

Perché ti serve Tkinter per le interfacce grafiche

Finora i tuoi programmi comunicano con l'utente tramite il terminale: testo che appare e testo che viene scritto. Con **Tkinter** puoi creare vere finestre grafiche con pulsanti, campi di testo, menu a tendina — come i programmi che usi ogni giorno.

Cos'è Tkinter?

Tkinter è la libreria per creare interfacce grafiche già inclusa in Python. Non devi installare nulla. Permette di creare finestre con tutti i classici elementi: etichette, pulsanti, caselle di testo, caselle di spunta, e molto altro.

La prima finestra

```
# Crea la finestra principale
root = tk.Tk()
root.title("La mia prima finestra")
root.geometry("400x300") # larghezza × altezza in pixel

# Avvia il "loop degli eventi" — mantiene la finestra aperta e risponde ai
click
root.mainloop()
```

Il metodo `mainloop()` è fondamentale: è il ciclo che mantiene la finestra in vita e reagisce a tutto ciò che l'utente fa (click, digitazione, ecc.).

I componenti principali (Widget)

In Tkinter, ogni elemento visibile si chiama **widget**.

Label — testo fisso

```
root = tk.Tk()

# Crea un'etichetta di testo
etichetta = tk.Label(root, text="Ciao, mondo!", font=("Arial", 16))
etichetta.pack(pady=20) # pack() posiziona il widget nella finestra

root.mainloop()
```

Button — pulsante cliccabile

```
def saluta():
    print("Ciao!") # Questa funzione viene chiamata al click

root = tk.Tk()

pulsante = tk.Button(root, text="Clicca qui!", command=saluta)
pulsante.pack(pady=20)

root.mainloop()
```

`command=saluta` indica quale funzione chiamare quando il pulsante viene premuto.

Entry — campo per inserire testo

```
root = tk.Tk()

# Crea un campo dove l'utente può scrivere
campo = tk.Entry(root, width=30)
campo.pack(pady=10)

def leggi_input():
    testo = campo.get() # Leggi il testo scritto dall'utente
    print(f"Hai scritto: {testo}")

pulsante = tk.Button(root, text="Leggi", command=leggi_input)
pulsante.pack()

root.mainloop()
```

Esempio completo: app di saluto

Metti insieme Label, Entry e Button per fare una piccola app interattiva:

```

def saluta():
    nome = campo_nome.get()
    if nome:
        etichetta_risultato.config(text=f"Ciao, {nome}!")
    else:
        etichetta_risultato.config(text="Inserisci un nome!")

root = tk.Tk()
root.title("App di saluto")
root.geometry("300x200")

# Istruzione per l'utente
tk.Label(root, text="Inserisci il tuo nome:").pack(pady=10)

# Campo di testo
campo_nome = tk.Entry(root, width=25)
campo_nome.pack()

# Pulsante
tk.Button(root, text="Saluta", command=saluta).pack(pady=10)

# Etichetta per mostrare il risultato (inizialmente vuota)
etichetta_risultato = tk.Label(root, text="", font=("Arial", 14))
etichetta_risultato.pack()

root.mainloop()

```

Posizionare i widget in griglia con `grid()`

`pack()` mette i widget uno sotto l'altro. Con `grid()` puoi posizzionarli in una **tabella** di righe e colonne — utile per form o calcolatrici:

```
root = tk.Tk()
root.title("Calcolatrice semplice")

# Riga 0: etichetta e campo per il primo numero
tk.Label(root, text="Primo numero:").grid(row=0, column=0, padx=5, pady=5)
campo_a = tk.Entry(root)
campo_a.grid(row=0, column=1)

# Riga 1: etichetta e campo per il secondo numero
tk.Label(root, text="Secondo numero:").grid(row=1, column=0, padx=5,
pady=5)
campo_b = tk.Entry(root)
campo_b.grid(row=1, column=1)

def calcola():
    try:
        a = float(campo_a.get())
        b = float(campo_b.get())
        etichetta_ris.config(text=f"Somma: {a + b}")
    except ValueError:
        etichetta_ris.config(text="Inserisci numeri validi!")

# Riga 2: pulsante che occupa entrambe le colonne
tk.Button(root, text="Somma", command=calcola).grid(row=2, column=0,
columnspan=2, pady=10)

# Riga 3: etichetta risultato
etichetta_ris = tk.Label(root, text="")
etichetta_ris.grid(row=3, column=0, columnspan=2)

root.mainloop()
```

Altri widget utili

```
root = tk.Tk()

# Text: area di testo su più righe (come un piccolo editor)
area = tk.Text(root, height=5, width=30)
area.pack()

# Checkbutton: casella di spunta
var_check = tk.BooleanVar()
check = tk.Checkbutton(root, text="Accetta i termini", variable=var_check)
check.pack()

# Radiobutton: scelta esclusiva tra opzioni
var_radio = tk.StringVar(value="mela")
for frutto in ["mela", "banana", "uva"]:
    tk.Radiobutton(root, text=frutto, variable=var_radio,
value=frutto).pack()

# Scale: cursore scorrevole
cursore = tk.Scale(root, from_=0, to=100, orient=tk.HORIZONTAL)
cursore.pack()

root.mainloop()
```

Finestre di dialogo

Tkinter ha anche finestre di dialogo già pronte — quelle che compaiono davanti alla finestra principale con un messaggio o una domanda:

```
from tkinter import messagebox

root = tk.Tk()

def mostra_info():
    # Finestra di informazione con un solo pulsante OK
    messagebox.showinfo("Info", "Operazione completata!")

def chiedi_conferma():
    # Finestra con pulsanti Sì e No
    risposta = messagebox.askyesno("Conferma", "Sei sicuro di voler uscire?")
    if risposta:
        root.quit() # Chiudi il programma

tk.Button(root, text="Mostra info", command=mostra_info).pack(pady=5)
tk.Button(root, text="Esci", command=chiedi_conferma).pack(pady=5)

root.mainloop()
```

Grafica con Turtle

Disegnare con la libreria Turtle in Python.

Qual è la funzione di Turtle nell'apprendimento

Turtle è il modo più semplice e divertente per vedere i risultati del tuo codice in modo visivo. Invece di stampare numeri sul terminale, disegni figure sullo schermo — e puoi vedere ogni riga del tuo programma "prendere vita".

È anche un ottimo strumento per capire come funzionano i cicli e la geometria.

Cos'è Turtle?

Turtle è una libreria già inclusa in Python — non devi installare nulla. Ti permette di controllare una "tartaruga" virtuale su uno schermo. La tartaruga ha una penna sotto il guscio: quando si muove, lascia una traccia. Tu dai i comandi e la tartaruga disegna.

L'idea viene dal linguaggio Logo degli anni '60, usato proprio per insegnare la programmazione ai bambini.

Il primo programma

```
t = turtle.Turtle()    # Crea la tartaruga

t.forward(100)          # Avanza di 100 pixel
t.right(90)             # Gira a destra di 90°
t.forward(100)          # Avanza ancora

turtle.done()           # Mantieni la finestra aperta
```

Comandi di movimento

```
t = turtle.Turtle()

t.forward(100)          # Avanza di 100 pixel (abbreviazione: t.fd(100))
t.backward(50)          # Arretra di 50 pixel (abbreviazione: t.bk(50))
t.right(90)             # Gira a destra di 90° (abbreviazione: t.rt(90))
t.left(90)              # Gira a sinistra di 90° (abbreviazione: t.lt(90))
t.goto(100, 50)         # Vai alle coordinate (x, y) – l'origine è al centro
t.home()                # Torna all'origine (0, 0) e guarda a destra
```


Alzare e abbassare la penna

```
t.penup()    # Alza la penna: la tartaruga si muove senza disegnare
t.pendown()  # Abbassa la penna: torna a disegnare
```

Utile per "saltare" in un altro punto senza lasciare tracce.

Personalizzare il disegno

```
t.pensize(3)          # Spessore della linea (in pixel)
t.pencolor("red")     # Colore della linea
t.fillcolor("yellow") # Colore di riempimento (usato con begin_fill /
end_fill)
t.speed(5)            # Velocità: 1=lenta, 10=veloce, 0=massima
(instantanea)
```

Disegnare figure geometriche

Quadrato

```
t = turtle.Turtle()

# Un quadrato ha 4 lati uguali e 4 angoli di 90°
for _ in range(4):
    t.forward(100) # Disegna un lato
    t.right(90)    # Gira di 90°

turtle.done()
```

Triangolo equilatero

```
t = turtle.Turtle()

# Un triangolo equilatero ha 3 lati e angoli esterni di 120°
for _ in range(3):
    t.forward(100)
    t.left(120)

turtle.done()
```

Cerchio

```
t = turtle.Turtle()

t.circle(50)    # Disegna un cerchio con raggio 50 pixel

turtle.done()
```

Stella a 5 punte

```
t = turtle.Turtle()
t.speed(0)

# Per una stella a 5 punte l'angolo esterno è 144°
for _ in range(5):
    t.forward(100)
    t.right(144)

turtle.done()
```

Riempire una figura con il colore

```
t = turtle.Turtle()

t.fillcolor("blue")    # Scegli il colore di riempimento
t.begin_fill()         # Da qui inizia il riempimento

for _ in range(4):     # Disegna il quadrato
    t.forward(100)
    t.right(90)

t.end_fill()           # Riempi la figura chiusa
turtle.done()
```

Personalizzare la finestra

```
schermo = turtle.Screen()
schermo.title("Il mio disegno")    # Titolo della finestra
schermo.bgcolor("black")           # Colore dello sfondo
schermo.setup(width=800, height=600) # Dimensioni in pixel

t = turtle.Turtle()
t.pencolor("white")    # Disegna in bianco su sfondo nero
t.forward(200)

turtle.done()
```

Esempio: spirale arcobaleno

Combina un ciclo, colori e geometria per creare qualcosa di bello:

```
t = turtle.Turtle()
t.speed(0)    # Velocità massima

colori = ["red", "orange", "yellow", "green", "blue", "purple"]

for i in range(100):
    t.pencolor(colori[i % len(colori)]) # Cambia colore ciclicamente
    t.forward(i * 2)                    # Ogni volta avanza un po' di più
    t.right(91)                          # 91° invece di 90° crea la
spirale

turtle.done()
```

Prova a modificare l'angolo (91°) o il numero di passi (100) per ottenere effetti diversi.